

Microsoft Tools for Back-End Development

Throughout this program, we will be using Microsoft tools, including C#, GitHub, GitHub Copilot and Microsoft Copilot, and VS Code.

Why Use C# for Back-End Development

- **Performance and Scalability:** C# is optimized for building high-performance, scalable back-end systems through its .NET runtime.
- **.NET Integration:** Tightly integrated with ASP.NET Core and other .NET libraries, C# makes building secure, robust back-end services easier.
- **Cross-Platform:** With .NET Core, C# supports development on Windows, macOS, and Linux, ideal for modern cloud-based solutions.
- **Strong Typing & OOP:** C#'s strong typing and object-oriented design create structured, maintainable code, reducing runtime errors.
- **Large Ecosystem:** Backed by a vast ecosystem and community support, C# offers tools and frameworks to simplify development and deployment.

Introduction to GitHub

GitHub is a platform for version control and collaboration, allowing developers to manage and share code effectively. It uses **Git**, a version control system, to track changes, collaborate in real time, and maintain a history of all modifications made to a project.

Key features include:

- **Version Control:** Tracks every change to your code, making it easy to revert to earlier versions and manage multiple project contributors.
- **Collaboration:** Facilitates teamwork through **pull requests** and **issues**, enabling code review, discussion, and project management.
- **Integration:** GitHub integrates seamlessly with tools like **Visual Studio Code** and cloud platforms, streamlining development and deployment workflows.
- **Open Source:** Hosts millions of open-source projects, providing a vast ecosystem of code and libraries that developers can contribute to or use.

GitHub is essential for managing code, fostering collaboration, and maintaining organized workflows in modern software development.

Why We're Using AI in This Program

AI is revolutionizing software development by automating repetitive tasks, enhancing productivity, and providing advanced tools for problem-solving. In this program, we're using AI-driven tools like GitHub Copilot and Microsoft Copilot to assist with coding, offer real-time code suggestions, and help you focus on complex tasks. This integration of AI will boost your efficiency and aid in learning advanced concepts more quickly.

GitHub Copilot vs. Microsoft Copilot

- **GitHub Copilot:** Embedded in Visual Studio Code, GitHub Copilot provides real-time code suggestions based on context. It helps you understand the logic behind the code and offers valuable in-line debugging support. GitHub Copilot will assist you in writing code and improve your problem-solving skills as you learn C#.
- **Microsoft Copilot:** Later in the program, we'll use Microsoft Copilot, a tool integrated with Microsoft 365. It's designed for code refinement, summarization, and explanation. Microsoft Copilot helps you document your code, identify potential issues, and refine your solutions—ensuring a deeper understanding of your work while remaining accessible and easy to use.

Introduction to Visual Studio Code

In this program, we'll use **Visual Studio Code (VS Code)**, a lightweight, flexible, and fast code editor from Microsoft. VS Code is ideal for back-end development and quick iterations. It supports multiple programming languages, such as **C#**, **HTML**, and **JavaScript**, making it perfect for back-end projects.

We're using VS Code because it offers:

- **Customization:** A vast extension marketplace to tailor the editor for specific needs.
- **Cross-Platform Support:** It works seamlessly on Windows, macOS, and Linux.
- **Version Control:** Built-in Git and GitHub integration for easy collaboration.

VS Code's simplicity and powerful features make it ideal for fast, efficient development.